

Type and Class

Tibor Gregorics and Balázs Pintér

Procedural vs. object-oriented paradigm

- ❑ **Procedural approach:** To solve a problem, we break it down into smaller tasks and organize the activities that address these tasks into independent units known as **procedures** (subprograms, macros, routines, functions). The process of solving the problem is defined by the chain of control transfers between the procedures (calls in the case of routines and functions).
- ❑ **Object-oriented approach:** We encapsulate parts of the data necessary for solving a problem into independent units known as **objects**, along with the associated behavior (the so-called methods). The process of solving the problem is defined by the handoffs of control between the methods of the objects (direct calls or signal transmissions).

Task

There are natural numbers between 0 and m in a non-empty array. What is the most common element in the array?

❑ Procedural solution: **maximum selection** and **counting**

- We count how many times the elements of the array occur in the array, and find the array element with the highest occurrence count.

❑ Object-oriented solution: **container object**

- We create a storage container where placing an element and querying the most frequent element are both fast. Then we place the elements in this container and query its most frequent element.

Type and Class

1st part

Executable specification

Tibor Gregorics and Balázs Pintér

Analysis

We specify the task using **variables**. The Ef describes what initial values these variables can have, while the Uf outlines our goal: what values should eventually appear in the variables, taking their initial values into account.

$S = (x:\mathbb{N}^n, m:\mathbb{N}, \text{elem}:\mathbb{N})$

Notation for the initial values of
Input variables: $x_0 \in \mathbb{N}^n, m_0 \in \mathbb{N}$

x has at least one element
elements are between 0 and m

$Q = (x = x_0 \wedge m = m_0 \wedge n \geq 1 \wedge \forall i \in [1..n]: x[i] \in [0..m])$

the input variables
retain their initial values

put the elements of x into the bag

$R = (x = x_0 \wedge m = m_0 \wedge b:\text{Bag} \wedge b = \bigcup_{i=1..n} [x[i]] \wedge \text{element} = \text{mostFreq}(b))$

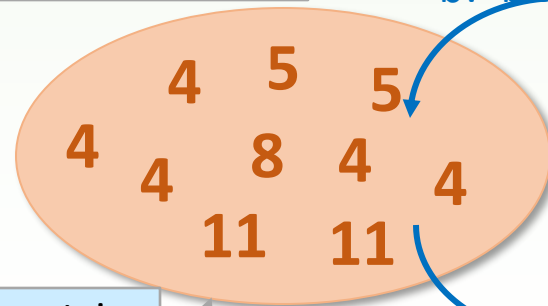
b is a helper variable

ask for the most frequent
element

$\bigcup$ an operation of combining a bag with a bag,
which has the empty bag as its neutral element($\emptyset$).
[e] it denotes a single-element bag containing e .

Variables and their types
 $x:\mathbb{N}^n \sim x$ is an array consisting
of natural numbers indexed
from 1 to n
 $m:\mathbb{N} \sim m$ is a natural number

$b =$



functions as a container
bag object

$b := b \bigcup [e]$

$e := \text{mostFreq}(b)$

Planning

$A = (x:\mathbb{N}^n, m:\mathbb{N}, b:\text{Bag}, \text{elem}:\mathbb{N})$

$Ef = (m = m_0 \wedge x = x_0 \wedge n \geq 1 \wedge \forall i \in [1 .. n]: x[i] \in [0 .. m])$

$Uf = (Ef \wedge b = \bigcup_{i=1..n} [x[i]] \wedge \text{elem} = \text{mostFreq}(b))$

Summation algorithm pattern

$s = \sum_{i=m..n} f(i)$

$f:[m..n] \rightarrow H$

$+: H \times H \rightarrow H$

left neutral element is 0

$s : H$

executable specification: not only describes what the task is, but also how it can be solved. The line between analysis and design becomes blurred.

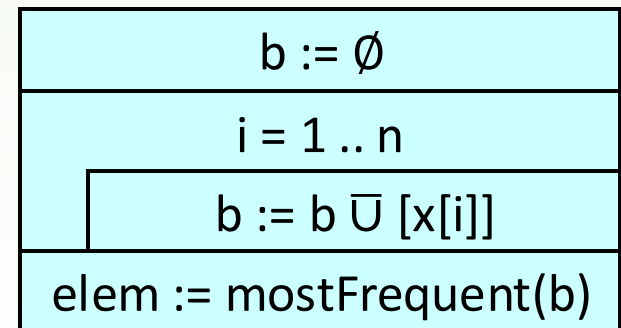
Analogy with summation:

same:

$$s = \sum_{i=m..n} f(i) \sim b = \bigcup_{i=1..n} [x[i]]$$

different:

result:	$s : H$	$\sim$	$b : \text{Bag}$
operation:	$H, +, 0$	$\sim$	$\text{Bag}, \bigcup, \emptyset$
elem:	$f(i)$	$\sim$	$[x[i]]$
enum:	$i = m .. n$	$\sim$	$i = 1 .. n$



Implementation without I/O

$b := \emptyset$
$i = 1 \dots n$
$b := b \cup [x[i]]$
$\text{elem} := \text{mostFrequent}(b)$

```
int n, m;  
...  
int x[] = new int[n];  
...  
Bag bag = new Bag(m);  
bag.erase();  
for( int i = 0; i < x.Count; ++i)  
{  
    bag.insert(x[i]);  
}  
int elem = bag.mostFrequent();
```

Diagram illustrating the mapping of C# code to mathematical set operations:

- `Bag` maps to `b:Bag`
- `new Bag()` maps to `b := ∅`
- `erase()` maps to `b := ∅`
- `insert(x[i])` maps to `b := b ∪ [e]`
- `mostFrequent()` maps to `mostFrequent(b)`

How do we give meaning to Bag and its operations?
We need to design the type of Bag and then implement it.

Type and Class

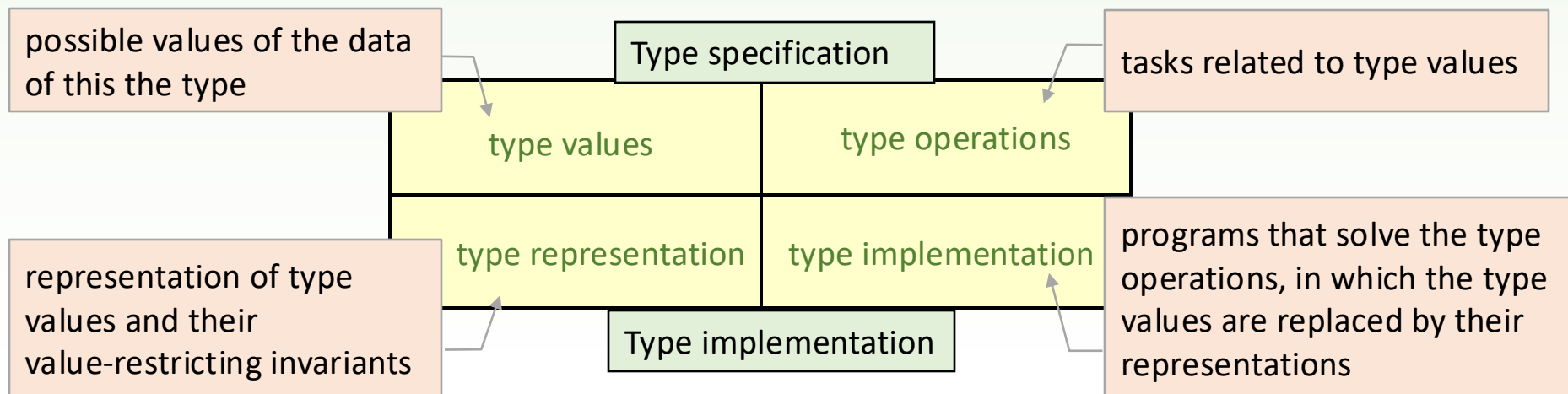
2nd part

Bag type

Tibor Gregorics and Balázs Pintér

Data type

- ❑ To define the type of a variable, both a type specification (how to use) and its implementation (how it's implemented) are needed.
- ❑ The type specification provides
 - the set of values that can be taken by the data: **type values**
 - operations that can be performed with type values: **type operations**
- ❑ The type implementation shows
 - how do we **represent** type values
 - The programs that **implement** these operations



Notable data types

Certain operations are not defined everywhere

- ❑ Natural number: $(\mathbb{N}, \{+, -, \cdot, /, \text{mod}\})$
- ❑ Integer: $(\mathbb{Z}, \{+, -, \cdot, /, \text{mod}\})$
- ❑ Real number: $(\mathbb{R}, \{+, -, \cdot, /\})$
- ❑ Boolean type: $(\mathbb{L}, \{\wedge, \vee, \neg\})$
- ❑ Character type: $(\mathbb{K}, \{\dots\})$
- ❑ Vector type: $(E^{m..n}, \{[.]\})$

one-dimensional arrays formed from the elements of set E, where the elements are indexed from m to n ($m, n \in \mathbb{Z}, m \leq n+1$); if $m=1$ then we use the E^n notation

- ❑ Matrix type: $(E^{l..n, k..m}, \{[. , .]\})$

two-dimensional arrays formed from elements of set E, where we index rows

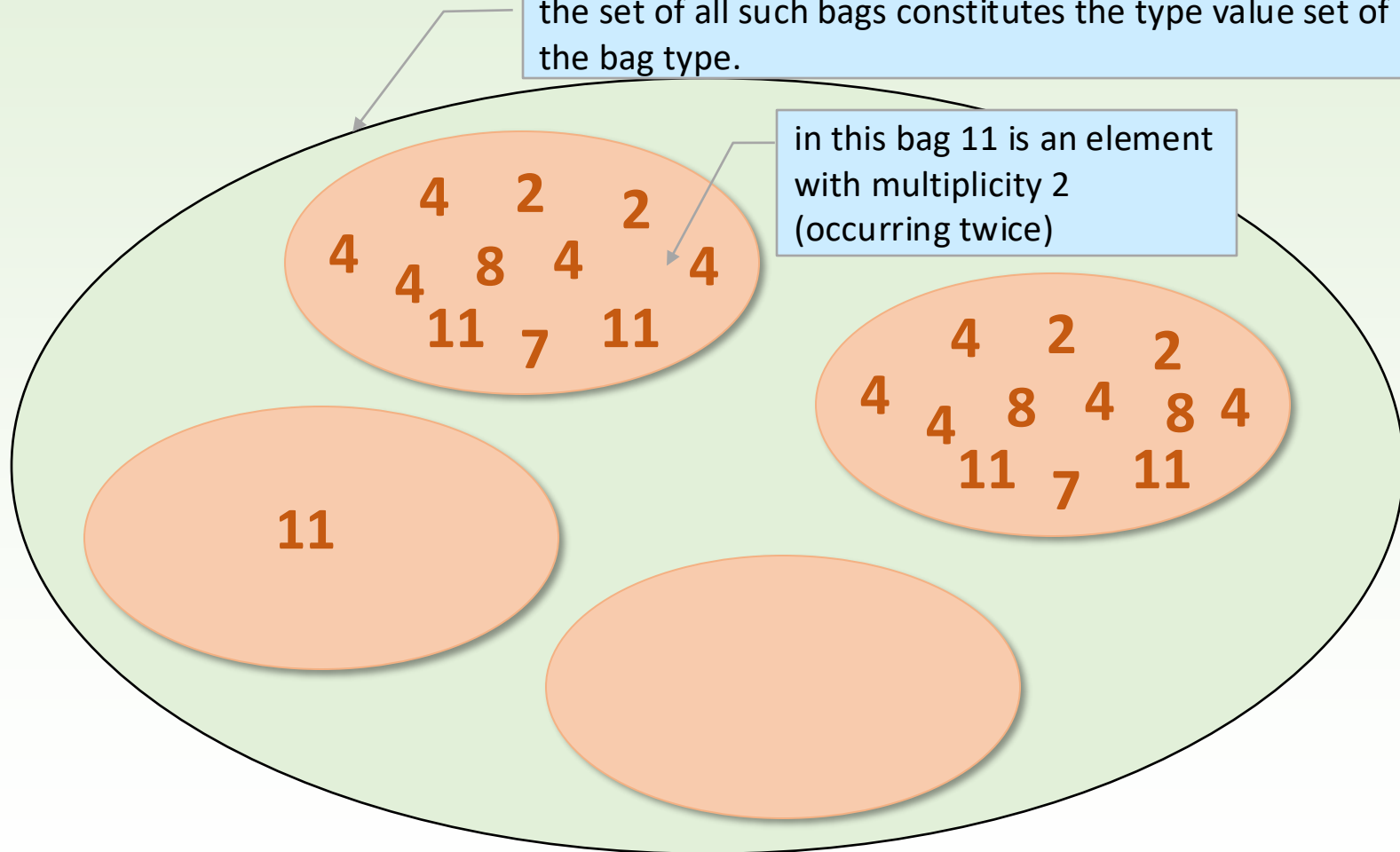
from l to n and columns from k to m ($l, n, k, m \in \mathbb{Z}, l \leq n+1, k \leq m+1$); if $l=k=1$ then $E^{n \times m}$

- ❑ Sequence type: $(E^*, \{[.], |. |, \oplus, \text{Add}(), \text{Remove}(), \dots\}), E^* = \bigcup_{n=0.. \infty} E^n$
- ❑ String type: $(\mathbb{S}, \{[.], |. |, \dots\})$ ahol $\mathbb{S} = \mathbb{K}^*$

We usually refer to types by the notation for the set of their type values.

Type value set for Bag

A variable with type Bag containing natural numbers has values that are bags containing natural numbers, and the set of all such bags constitutes the type value set of the bag type.



Operations of the Bag type

Empty the bag:

$b := \emptyset$

$b:\text{Bag}$

Give error if
 $e \notin [0 .. m]$

Put an item in the bag.:

$b := b \cup [e]$

$b:\text{Bag}, e:\mathbb{N}$

give error if the
bag is empty

Most common element of the bag:

$e := \text{mostFreq}(b)$ $b:\text{Bag}, e:\mathbb{N}$

Representation of Bag type

b:

4 2 2
4 4 8 4 4
11 7 11

A bag can contain integers between 0 and m , whose frequencies are stored in an array with an index range of 0 to m .

vec:

0	1	2	3	4	5	6	7	8	9	10	11	...	m
0	0	2	0	5	0	0	1	1	0	0	2	...	0

 : $\mathbb{N}^{0..m}$

max:

4

 : $\mathbb{N}$

we store the most common item

Type invariant:

$\max \in [0..m] \wedge$

$\vec{vec}[\max] = \max_{i=0}^m \vec{vec}[i]$

the relationship between the data used to represent a type value

useful: $\vec{vec}[\max]=0$
shows that the bag is empty

When is a representation good?

If it represents any type value (bag) with a combination of elements (vec-max pair) that **satisfies the type invariant**. Furthermore, the set of elements that satisfy the type invariant (vec-max pair) represents a **unique** type value (bag).

Implementation of bag type

Error if $e \notin [0..m]$

$b := b \cup \{e\}$

$0 \leq e \leq m$

$\text{vec}[e] := \text{vec}[e] + 1$

$\text{vec}[e] > \text{vec}[\text{max}]$

$\text{max} := e$

—

ERR

When is the implementation good?

If it provides a program for every type operation where the type values (bags) are represented by elements (vec-max pairs) that satisfy the type invariant.

type invariant:

$\text{max} \in [0..m] \wedge$

$\bigwedge_{i=0}^m$

$\text{vec}[\text{max}] = \max_{i=0} \text{vec}[i]$

$i=0$

$b := \emptyset$

$i = 0 \dots m$

$\text{vec}[i] := 0$

$\text{max} := 0$

max could be anything between $0..m$ and still ensure the type invariant, as $\forall i \in [0..m]: \text{vec}[i] = 0$

due to type invariant

there's no need to check the type invariant

$e := \text{mostFreq}(b)$

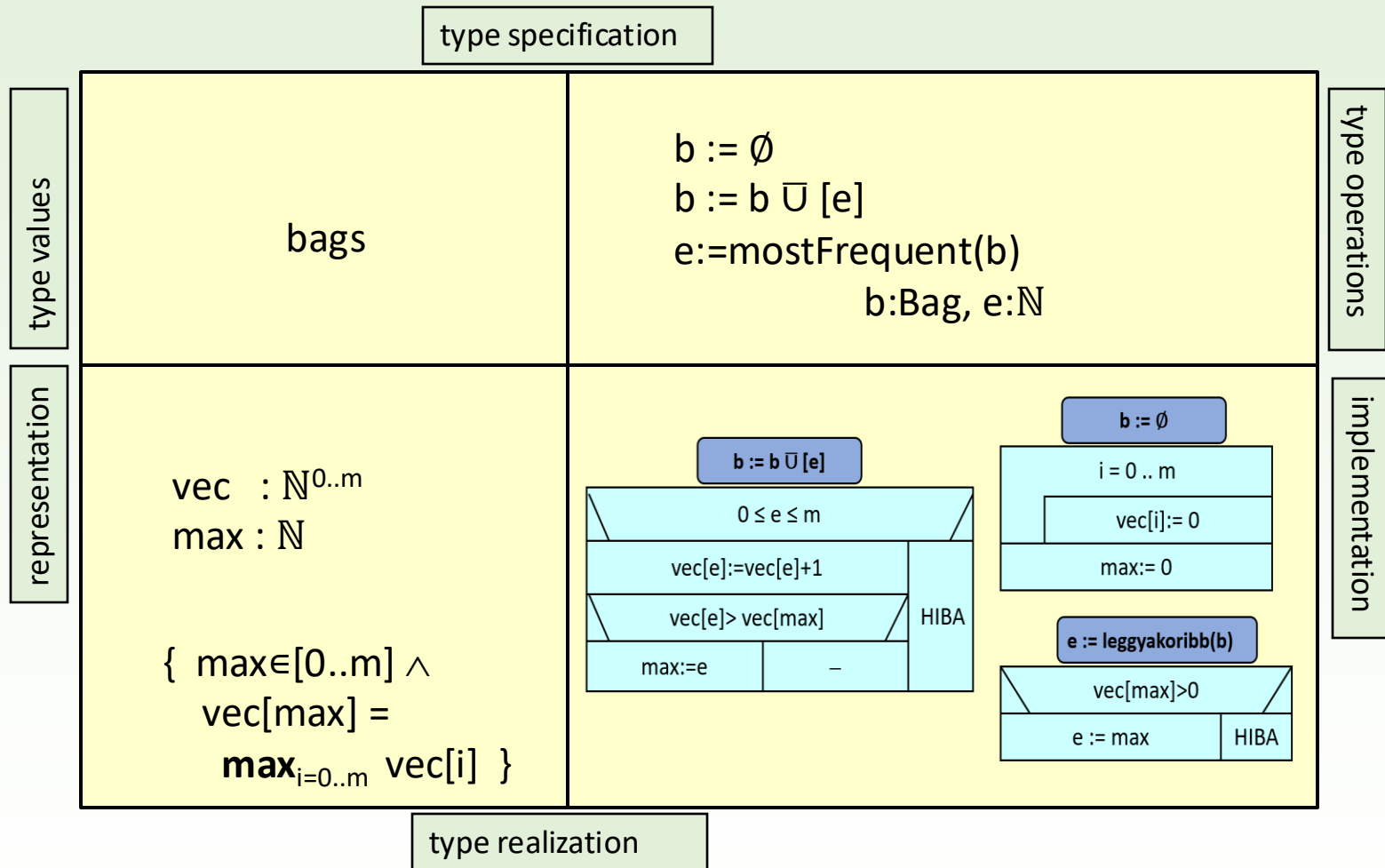
$\text{vec}[\text{max}] > 0$

$e := \text{max}$

ERR

error if the bag is empty.

Bag type



Type and class

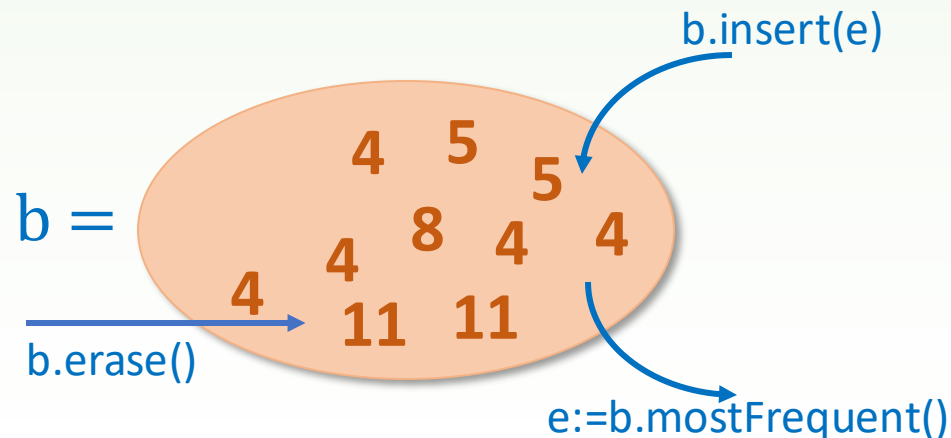
3rd part

Bag class and its code

Tibor Gregorics and Balázs Pintér

Object

- ❑ An object is defined as part of a software solution to a task identified as an **independent entity**, which includes the **data and operations** responsible for that part.
- ❑ Once we create an object (instantiate it), we can only reference it indirectly through a variable. This **object variable** usually contains the address of the memory area that is allocated during the program's execution (when the object is instantiated) so that we can store the object's data members there.



Class

- ❑ The class provides a **blueprint** for the structure and behavior of an object, that is,
 - lists the object's **attributes** by specifying their names, types, and visibility (private (-), protected (#), or public (+)), along with any potential type invariants
 - provides the callable **methods** for the object (member function, operation) along with their names, parameter lists, return value types, bodies, and visibility.
- ❑ The class essentially defines the **type of the object**: we create the object based on its class, meaning we **instantiate** it.
- ❑ Multiple objects can be instantiated from a class: each object has the attributes and methods defined by the class description.

Type and Class

- In object-oriented design, we define unique or so-called **custom types** as classes.

Bag type:

type specification

bags

$b := \emptyset$
 $b := b \cup [e]$
 $e := \text{mostFreq}(b)$
 $b : \text{Bag}, e : \mathbb{N}$

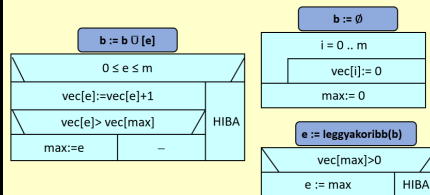
type operations

type values

representation

$\text{vec} : \mathbb{N}^{0..m}$
 $\text{max} : \mathbb{N}$
 $\{ \text{max} \in [0..m] \wedge$
 $\text{vec}[\text{max}] =$
 $\max_{i=0..m} \text{vec}[i] \}$

type implementation



implementation

Bag class:

$\text{max} \in [0..m] \wedge$
 $\text{vec}[\text{max}] = \max_{i=0..m} \text{vec}[i]$

Bag

$\text{vec} : \text{int}[0..m]$
 $\text{max} : \text{int}$
 $+$ erase() : void
 $+$ insert(e:int) : void
 $+$ mostFrequent() : int

for i=0..m loop vec[i] := 0 endloop
max := 0

if vec[max] ≤ 0 then error endif
return max

if not (0 ≤ e ≤ m) then error endif
++vec[i]
if vec[e] > vec[max] then max := e endif

Class of bag type in Java

```
public class Bag
{
    private int[] vec;
    private int max;

    public void erase()
    {
        for (int i = 0; i < vec.length; ++i) vec[i] = 0;
        max = 0;
    }
    public void insert(int e)
    {
        if ( e < 0 || e >= vec.length ) return; // error handling is required
        if ( ++vec[e] > vec[max] ) max = e;
    }
    public int mostFrequent()
    {
        if( vec[max] == 0 ) return -1; // error handling is required
        return max;
    }
}
```

Type name

Only usable within the class

Fields based on the type representation.

Can be called from outside the class

Methods from type operations

array length

The fields (vec, max) used in the body of the methods are the fields of the bag object on which the method is called: in the case of b.insert(3), it refers to the vec and max members of the b object.

1

A key feature of object-oriented languages is **encapsulation**: we can define the data needed to implement a specific task and the program components that manipulate that data separately from the rest of the program.

Constructor

The **instantiation** (creation) of the object is performed by a special method, the constructor: it allocates memory for the object's fields (if a field is itself an object reference, it also instantiates it) and assigns initial values to them.

There is no return type.
Name: the name of its class.

If we don't define a constructor, a **default constructor** is provided for us (with no parameters and an empty body).

```
public class Bag
{
    private int[] vec;
    private int max;

    public Bag(int m)
    {
        vec = new int[m+1];
        max = 0;
    }
    ...
}
```

The `Bag b = new()` statement would call the empty constructor, which does not provide an opportunity to specify the length of the array representing the bag (the value of `m`), nor does it guarantee that the fields are initialized in accordance with the invariant.

Let's create a constructor that takes the length (`m`) of the `vec` array as a parameter so it can allocate its storage space.
`Bag b = new(35);`

Moreover, the new bag's data members must be initialized so they satisfy the type invariant. For this, it's enough to set the `max` value to zero while all elements of the array are initialized to zero by default, thereby instantiating an empty bag.

Using members of an object

```
class Bag
{
    private int[] vec;
    private int max;
    public Bag(int m)        { ... }
    public void erase()      { ... }
    public void insert(int e) { ... }
    public int mostFrequent() { ... }
}
```

```
Bag b1 = new(5);
Bag b2 = new(23);
b1.erase();
b2.insert(5);
int a = b2.mostFrequent();
b1.max = 0;
b1.vec[5]++;
```

When we want to use a member (field or method) of an object, we need to write the object (more precisely, the variable referencing it) before the member we intend to use. The object written before the method will be a special extra parameter of the method.

We can only refer to an object's hidden (private, protected) members within the body of the object's methods; elsewhere, these cannot be used directly.

2

An important feature of object-oriented languages is information hiding: limiting the visibility of the elements contained within a unit. (Generally, data members are hidden, and we can only access their values indirectly through public methods.)

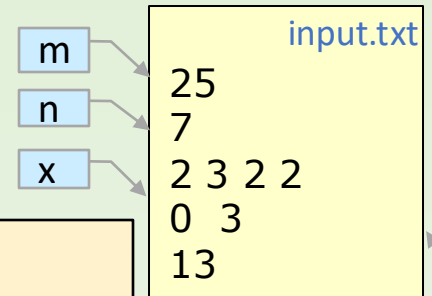
Type and class

4th part

Implementation

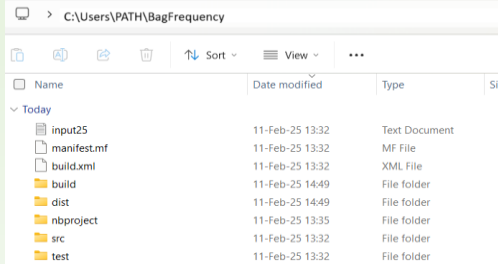
Tibor Gregorics and Balázs Pintér

Reading inputs



How do we read integers from a text file?

Don't forget to import the needed packages



Place your text file in the root directory of your java project. For example, if your project is located in /MyProject/ you can place a file directly in /MyProject/ folder. In this case when using the file you directly write the file name and its extension. In this example "input.txt"

```
import java.io.BufferedReader;
import java.io.FileReader;
import java.util.Scanner;
public class Main
{
    .
    .
    String filename = "input25.txt";
    Scanner sc = new Scanner(new BufferedReader(
    while (sc.hasNextInt())){
        System.out.println (sc.nextInt());
    }
    sc.close();
    .
    .
}
```

We use Scanner to read integers from the text file. Scanner sc = new Scanner (...) creates a scanner object to read from the file. BufferedReader is used if you want to read the file line by line

sc.hasNextInt():

Checks if the next token in the file is an integer.

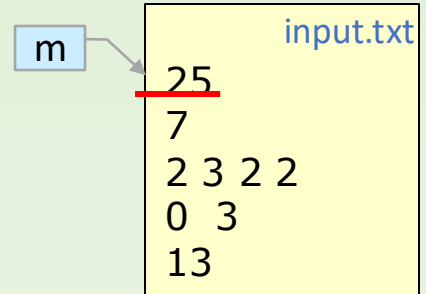
sc.nextInt():

Reads the next integer from the file and assigns it to number.

sc.close():

Closes the Scanner when done to free resources.

Implementation



```
String filename = "input25.txt";
Scanner sc = new Scanner(new BufferedReader(new FileReader(filename)));

int m = sc.nextInt();
Bag bag = new Bag(m);

while (sc.hasNextInt()){
    bag.insert(sc.nextInt());
}

sc.close();

System.out.println ("The most frequent element: " + bag.mostFrequent());
```

Structure of the solution

solution: Frequency
project: Frequency
main.java
Bag.java

package frequency

```
public class Program
{
    Public static void Main()
    {
        ...
    }
}
```

Static method, no
object is needed to call

Main.java

```
public class Bag
```

```
private int[] vec;
private int    max;

public Bag(int m){...}
public void erase(){...}
public void insert(int e){...}
public int mstFrequent(){...}
}
```

Bag.java

Main

Exception handling: If an instruction (e.g., a method call) detects an error, it should throw an exception; this interrupts the program's execution. If it happens within a try block, we have the opportunity to handle the cause of the exception in the catch blocks placed after the block – where control is transferred in such cases.

```
package frequency;
import java.io.BufferedReader;
import java.io.FileReader;
import java.util.Scanner;
public class Main
{
    public static void main(String[] args)
    {
        try
        {
            String filename = "input25.txt";
            Scanner sc = new Scanner(new BufferedReader(new FileReader(filename)));
            int m = sc.nextInt();
            Bag bag = new Bag(m);
            while (sc.hasNextInt())
            {
                try { bag.insert(sc.nextInt()); }
                catch (Bag.IllegalElementException ex) {
                    System.out.println("The element of the bag must be in [0.." + m + "].");
                }
            }
            System.out.println("The most frequent element: " + bag.mostFrequent());
        }
        catch (java.io.FileNotFoundException e) {
            System.out.println("Input file does not exist");
        }
        catch (Bag.NegativeSizeException e) {
            System.out.println("Upper limit of the input natural numbers cannot be negative.");
        }
        catch (Bag.EmptyBagException e) {
            System.out.println("There is no most frequented element");
        }
    }
}
```

exception monitoring

throws an exception if the text file is not found

throws an exception if m is negative

throws an exception if inserted item is not between 0 and m

throws an exception if b is empty

catching and handling exceptions

Main.java

Defining exceptions

```
package frequency;
```

```
public class Bag {
```

```
    public class NegativeSizeException extends RuntimeException {}
```

```
    public class IllegalElementException extends RuntimeException {}
```

```
    public class EmptyBagException extends RuntimeException {}
```

```
    protected int[] vec;
```

```
    protected int max;
```

```
    public Bag(int m){    }
```

```
    public void erase(){    }
```

```
    public void insert(int e){    }
```

```
    public int mostFrequent() {    }
```

```
}
```

Errors that occur during the operation of a bag-type object.

We define the error cases as "exceptions" through derivation.

Bag.java

Throwing exceptions

Throws an exception if the value of `m` is negative, and the instantiation of the object is interrupted.

```
public Bag(int m){
    if (m < 0) throw new NegativeSizeException();
    vec = new int[m+1]; // all elements initialized to 0 by default
    // for (int i = 0; i <= m; ++i) vec[i]=0;
    max = 0;
}
public void erase(){
    for (int i = 0; i < vec.length; ++i) vec[i] = 0;
    max = 0;
}
public void insert(int e){
    if (e < 0 || e >= vec.length) throw new IllegalElementException();
    if (++vec[e] > vec[max]) max = e;
}
public int mostFrequent() {
    if (vec[max] == 0) throw new EmptyBagException();
    return max;
}
```

Error detection is separate from error handling.

throws an exception if the parameter value is not between 0 and `m`.

throws an exception if bag is empty.

Bag.java

Handling Exceptions

```
try
{
    .....
    while (sc.hasNextInt())
    {
        try { bag.insert(sc.nextInt()); }
        catch (Bag.IllegalElementException ex)
        {
            System.out.println("The element of the bag must be in [0..{" + m + "}].");
        }
    }
    .....
}
catch (java.io.FileNotFoundException e)
{
    System.out.println("Input file does not exist");
}
catch (Bag.NegativeSizeException e)
{
    System.out.println("Upper limit of the input natural numbers cannot be negative.");
}
catch (Bag.EmptyBagException e)
{
    System.out.println("There is no most frequented element");
}
```

After handling the error, the loop containing the try block continues.

If there is a catch block that matches the exception detected in a try block, then the program control is transferred here.

Main.java